

StationarityToolkit: Comprehensive Time Series Stationarity Analysis in Python

Bhanu Suraj Malla¹ and Yuqing Hu¹

¹ Georgia Institute of Technology, United States Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [✉](#)

Submitted: 16 March 2026

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

Summary

Time-series stationarity is a property that statistical characteristics such as trend, variance, seasonality remain constant over time. It is considered fundamental to many forecasting and analysis methods. Different tests detect different types of non-stationarity: structural breaks or deterministic trends, clustered or time-dependent variance, stochastic or deterministic seasonality. A series might pass one test while failing another; single-test approaches seldom distinguish between conceptually different types of non-stationarity that require different types of tests and transformations.

StationarityToolkit addresses this by providing a comprehensive Python library that runs 10 statistical tests across three categories: trend (4 tests), variance (4 tests), and seasonality (2 tests). Rather than a binary stationary/non-stationary verdict, users receive detailed diagnostics with actionable notes for each detection. The toolkit automatically infers the frequency of the data provided (requires datetime index), provides clear interpretations with test statistics and p-values, and supports an iterative test-transform-retest workflow essential for real-world data sets.

Statement of Need

Stationarity testing is one of the critical preprocessing steps for time series analysis, but it is not a single question with a single answer. Non-stationarity can manifest as unit roots, deterministic trends, structural breaks, variance instability, volatility clustering, or seasonal patterns - each requiring different transformations. Practitioners need to identify *which type* of non-stationarity exists to choose the right remedy: differencing for unit roots ([Dickey & Fuller, 1979](#)), detrending for deterministic trends, Box-Cox ([Box & Cox, 1964](#)) for variance instability, or seasonal differencing for periodic patterns.

This gap becomes particularly challenging when transformations interact unpredictably. Differencing to remove trend can introduce variance non-stationarity; variance stabilization can be undone by subsequent differencing. Without comprehensive testing after each transformation, practitioners cannot verify whether their preprocessing actually achieved stationarity or introduced new problems. An iterative test-transform-retest workflow is essential, but orchestrating this across multiple libraries and tests is tedious and error-prone.

Potential users include data scientists, econometricians, and researchers working with time series data who need to prepare data for different use cases including ARIMA/SARIMA models, VAR analysis, Granger causality tests, or machine learning applications where stationarity improves generalization. The toolkit is particularly valuable for practitioners who need to understand *what type* of non-stationarity exists in their data.

State of the Field

Several Python packages address aspects of time series stationarity testing. The statsmodels library (Seabold & Perktold, 2010) provides individual unit root tests (ADF (Dickey & Fuller, 1979), KPSS (Kwiatkowski et al., 1992), Zivot-Andrews (Zivot & Andrews, 1992)), seasonal decomposition (STL (Cleveland et al., 1990)), and heteroscedasticity diagnostics (White's test, ARCH test). The arch package (Sheppard, 2017) offers unit root tests (ADF, KPSS, Phillips-Perron (Phillips & Perron, 1988), Zivot-Andrews) and ARCH/GARCH models (Engle, 1982) for volatility modeling, though its primary focus is on fitting volatility models rather than comprehensive stationarity diagnostics. The scipy library (Virtanen et al., 2020) offers variance comparison tests (Levene (Levene, 1960), Bartlett (Bartlett, 1937)). The pmdarima library (Smith, 2015) includes stationarity tests (ADF, KPSS, Phillips-Perron) primarily as preprocessing for auto-ARIMA model selection rather than as standalone diagnostic tools. In all cases, users must manually run tests from different libraries, interpret potentially conflicting results, and determine which transformations to apply.

Table 1 summarizes the coverage of each library across stationarity dimensions.

	statsmodels	arch	scipy	pmdarima	StationarityToolkit
Unit root (ADF, KPSS, PP)	partial	✓	×	✓	✓
Structural break (ZA)	✓	✓	×	×	✓
Variance (Levene, Bartlett)	×	×	✓	×	✓
White / ARCH test	✓	×	×	×	✓
Seasonal decomposition	✓	×	×	×	✓
Seasonal stationarity test	×	×	×	×	✓
Auto frequency inference	×	×	×	×	✓
Contextual period selection	×	×	×	×	✓

Note: statsmodels has ADF and KPSS but not Phillips-Perron (available in arch). It provides White's test (het_white) and ARCH test (het_arch) as regression diagnostics, and STL for decomposition — but none are integrated into a stationarity testing workflow. scipy provides Levene and Bartlett as generic variance comparison functions without time-series segmentation.

StationarityToolkit differs by integrating testing across all the stationarity dimensions - trend, variance, and seasonality - in a single call with a report that summarizes test outcome, actionable notes, caveats, and their statistical interpretation. It goes one step further to infer time-series frequency (requires datetime index) before testing for seasonal non-stationarity to maintain intuitiveness in its results. This approach also reveals test limitations in its notes (e.g., Zivot-Andrews detecting “breaks” in smooth trends, ARCH triggering on auto-correlation) that single-test approaches miss.

65 The design philosophy of the toolkit prioritizes transparency over prescription; the toolkit shows
66 users what's happening in their data rather than making transformation decisions for them.
67 This is because transformation effectiveness could vary unpredictably across datasets, and
68 across use cases as demonstrated in the package documentation where identical transformations
69 produce opposite variance outcomes on synthetic versus real data.

70 Software design

71 StationarityToolkit is implemented as a pure Python package with dependencies on
72 numpy (Harris et al., 2020), pandas (McKinney, 2010), scipy (Virtanen et al., 2020), arch
73 (Sheppard, 2017), and statsmodels (Seabold & Perktold, 2010). The architecture separates
74 test implementation (in tests/ modules organized by trend, variance, and seasonality),
75 result formatting and output generation (results.py), and the main detection entry point
76 (toolkit.py). This design enables community contributions, reliably expanding the suite of
77 tests while maintaining consistent user experience.

78 The toolkit evolved significantly from its initial design. Early versions (0.x) attempted to
79 provide both testing and automated transformations with inverse functions, but this approach
80 proved problematic as the effectiveness of transformations was found to vary unpredictably
81 across datasets, and their needs vary by use case; this meant automated decisions obscured
82 what was actually happening to the data. Version 1.x and the following 2.x represented a
83 fundamental redesign, pivoting to a scaled-down pure diagnostic analysis tool with an expanded
84 test suite grounded in statistical theory. This design choice - diagnostics over automation -
85 was based on the reflection about the reality that practitioners need to understand their data's
86 specific non-stationarity characteristics to make informed decisions for their use case.

87 Key design decisions include:

- 88 1. **Comprehensive testing by default:** All of the 10 statistical tests are run in a single call
89 eliminating the cognitive burden on deciding which tests to run. This also ensures that
90 the users don't miss critical information that they would otherwise miss.
- 91 2. **Structured output with actionable notes:** Each test returns not just pass/fail but also
92 concise notes on the test, its caveats and suggestions (e.g. "Unit root detected - consider
93 differencing", "Deterministic trend detected - stationary after detrending").
- 94 3. **Contextual seasonality detection:** Seasonal tests automatically determine appropriate
95 periods to test based on time series frequency (e.g. testing for weekly, monthly, and
96 yearly seasonality in daily data), eliminating the need for manually specifying periods to
97 test. This requires the time series to have datetime index.
- 98 4. **2-pronged reporting:** The report() method returns results as a pandas DataFrame
99 and optionally writes a markdown report when a filepath is provided, supporting both
100 interactive analysis and documentation.
- 101 5. **Intuitive trend tests:** All unit root tests (ADF (Dickey & Fuller, 1979), KPSS
102 (Kwiatkowski et al., 1992), Phillips-Perron (Phillips & Perron, 1988)) run to test both
103 constant-only and constant-plus-trend parameters, intuitively determining whether
104 non-stationarity is due to unit roots or deterministic trends.

105 The implementation prioritizes correctness and interpretability. Tests run sequentially and the
106 typical execution time for all tests on a 1000-row series is under 2 seconds on modern hardware.

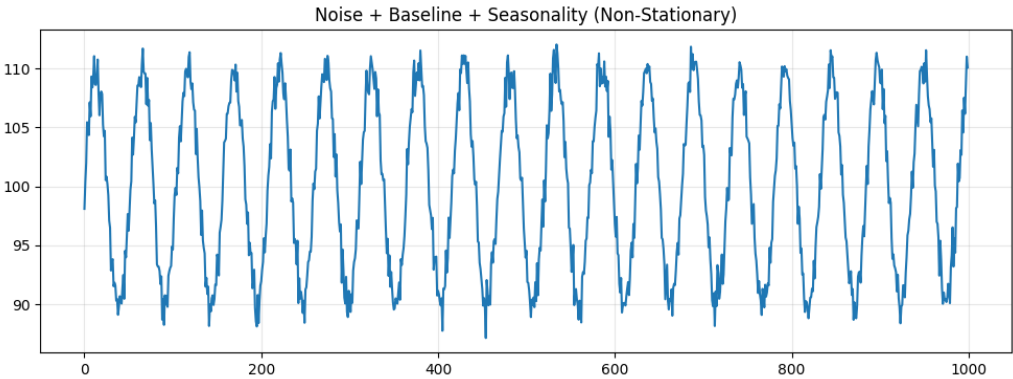


Figure 1: Example input: synthetic time series with noise, baseline, and seasonality.

Trend Stationary:

Yes

No

Variance Stationary:

No

Yes

Seasonal Stationary:

No

Yes

	Test Type	Test	Result	Statistic	P-value	Note	Interpretation
0	Trend	Augmented Dickey-Fuller (ADF) Test	Stationary	-14.4895	0.0000	Stationary around constant mean	H0: Unit root. ADF-c p=0.0000 < 0.05. ADF-t p=0.0000 < 0.05. Reject H0.
1	Trend	Kwiatkowski-Phillips-Schmidt-Shin (KPSS) Test	Stationary	0.0099	0.1000	Stationary around constant mean	H0: Stationary. KPSS-c p=0.1000 > 0.05. KPSS-t p=0.1000 > 0.05. Fail to reject H0.
2	Trend	Phillips-Perron (PP) Test	Stationary	-6.5116	0.0000	Stationary around constant mean	H0: Unit root. PP-c p=0.0000 < 0.05. PP-t p=0.0000 < 0.05. Reject H0.
3	Trend	Zivot-Andrews Test for Structural Breaks	Stationary	-14.7602	0.0000	Stationary with structural breaks: level shift at obs 721, trend shift at obs 264, level+trend shift at obs 721. Note: ZA detects discrete breaks, not smooth trends. Breaks may be spurious in noise.	H0: Unit root with no break. ZA-c p=0.0000, ZA-t p=0.0010, ZA-ct p=0.0007 < 0.05. Reject H0 - stationary.
4	Variance	Levene's Test for Variance Homogeneity	Stationary	0.4526	0.7155	Constant variance across time	H0: Equal variances across segments. Levene p=0.7155 > 0.05. Fail to reject H0.
5	Variance	Bartlett's Test for Variance Homogeneity	Stationary	0.3359	0.9531	Constant variance across time	H0: Equal variances across segments (assumes normality). Bartlett p=0.9531 > 0.05. Fail to reject H0.
6	Variance	White's Test for Heteroskedasticity	Stationary	0.5885	0.7451	Constant variance across time	H0: Homoskedasticity. White p=0.7451 > 0.05. Fail to reject H0.
7	Variance	ARCH Test for Conditional Heteroskedasticity	Non-stationary	962.2527	0.0000	Volatility clustering detected - consider GARCH modeling (may trigger on trend/seasonality if present)	H0: No ARCH effects. ARCH p=0.0000 <= 0.05. Reject H0.
8	Seasonal	ACF/PACF Peak Detection	Non-stationary	0.9707	0.0000	Seasonality detected (periods: 52) - consider seasonal differencing (may trigger on trend/variance)	H0: No seasonality. Ljung-Box p=0.0000 < 0.05. Reject H0.
9	Seasonal	STL Decomposition	Non-stationary	62.4157	0.0000	Significant seasonal component detected (period 52) - consider seasonal differencing	H0: No seasonality. F-stat p=0.0000 <= 0.05. Reject H0.

Figure 2: Example output: StationarityToolkit summary and detailed test results DataFrame.

Research Impact Statement

StationarityToolkit addresses a genuine gap in the Python ecosystem; while individual stationarity tests exist in various libraries, no tool provides comprehensive integrated testing across trend, variance, and seasonality dimensions with actionable notes and caveats. The package has been publicly available on PyPI since 2023. The toolkit is used in a companion study evaluating whether stationarity transformations improve time series forecasting accuracy (currently under review), where it serves as the diagnostic engine for testing stationarity before and after transformations.

The toolkit's research significance lies in its ability to reveal test limitations and cross-contamination effects (e.g. variance non-stationarity emerging as a result of differencing to allay trend non-stationarity) that single-test approaches miss. The comprehensive documentation, including validation on synthetic as well as real data (examples/detailed_usage.ipynb), provides researchers with a context for when and why different tests succeed or fail. This transparency is particularly valuable when reproducibility is important - where understanding why a transformation was chosen matters as much as the transformation itself.

The package demonstrates credible significance through its complete implementation of established statistical tests, clear documentation with reproducible examples, and active maintenance. It is designed for immediate integration into research workflows, with pandas DataFrame outputs that integrate seamlessly with existing analysis pipelines and markdown export for documentation.

AI Usage Disclosure

The toolkit's core design, tests write-up, examples and this paper were all conceived and written by human author. The specific use of AI is as follows:

Tool use: Claude (Anthropic, versions 3.5 Sonnet and Opus 4) was used for refining code, docs.

Nature and scope of assistance: AI assisted with removing redundant code (written by human), changing variable names (for intuitiveness), adding doc-strings, and documentation (refining README and notebooks inside examples/). Also, the idea of using dataclass for results.py was assisted by AI with focus to improve the result synthesis for users.

Confirmation of review: All AI-generated content was reviewed, edited, and validated by the human author. Core design decisions were made by the human author based on domain expertise in time series analysis. Test implementations were validated against established libraries (statsmodels, arch, scipy) and on synthetic data with known properties. The human author takes full responsibility for the accuracy, correctness, and scientific validity of all submitted materials.

References

- Bartlett, M. S. (1937). Properties of sufficiency and statistical tests. *Proceedings of the Royal Society A*, 160(901), 268–282.
- Box, G. E. P., & Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2), 211–252.
- Cleveland, R. B., Cleveland, W. S., McRae, J. E., & Terpenning, I. (1990). STL: A seasonal-trend decomposition procedure based on loess. *Journal of Official Statistics*, 6(1), 3–33.
- Dickey, D. A., & Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, 74(366a), 427–431.
- Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation. *Econometrica*, 50(4), 987–1007.
- Harris, C. R., Millman, K. J., Walt, S. J. van der, & others. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Kwiatkowski, D., Phillips, P. C. B., Schmidt, P., & Shin, Y. (1992). Testing the null hypothesis of stationarity against the alternative of a unit root. *Journal of Econometrics*, 54(1-3), 159–178.
- Levene, H. (1960). Robust tests for equality of variances. In I. Olkin (Ed.), *Contributions to probability and statistics* (pp. 278–292). Stanford University Press.
- McKinney, W. (2010). Data structures for statistical computing in python. *Proceedings of the 9th Python in Science Conference*, 445, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- Phillips, P. C. B., & Perron, P. (1988). Testing for a unit root in time series regression. *Biometrika*, 75(2), 335–346.
- Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with python. *9th Python in Science Conference*.
- Sheppard, K. (2017). *Arch: ARCH models in python*. Zenodo. <https://doi.org/10.5281/zenodo.593254>
- Smith, T. G. (2015). *Pmdarima* (Version 2.1.1). <https://github.com/alkaline-ml/pmdarima>

- 170 Virtanen, P., Gommers, R., Oliphant, T. E., & others. (2020). SciPy 1.0: Fundamental
171 algorithms for scientific computing in python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
172
- 173 Zivot, E., & Andrews, D. W. K. (1992). Further evidence on the great crash, the oil-price
174 shock, and the unit-root hypothesis. *Journal of Business & Economic Statistics*, 10(3),
175 251–270.

DRAFT